GPTSort: An Earth-Shattering, Paradigm-Shifting New Sorting Algorithm

Stefan Muller
University of Connecticut

ABSTRACT

We present a new AI-enabled sorting algorithm, GPTSort. In contrast to conventional sorting algorithms, GPTSort does not require implementers to have any knowledge of integer sorting. When run using the GPT 4.1 model, GPTSort can sort 10 integers of up to 7 digits in under 2 seconds, and is correct almost all of the time.

1 INTRODUCTION

Sorting algorithms, which take a list of comparable items called *keys*, and return a permutation of the input list sorted by increasing key, are some of the first algorithms that undergraduate computer science students learn. But you don't need to know what they are to appreciate this paper or, apparently, at all anymore. *Algorithms* are predetermined procedural strategies for solving problems, but apparently you don't need to know that anymore either. You don't need to know these things because Artificial Intelligence (AI) has now advanced to the point where we can just have it know things for us. So all of these things we used to know are useless because people knew these things in the past. Things people knew in the past used to be called *scholarship*. We now call it *training data*. That's on the borderline of things you need to know. But probably not for long. Because soon AI will know that for us too. It used to be that people thought that because AI was the future, it was important to study how to build and maintain AI tools to make that future the present, but now that it's the present, those things people knew in the past (*training data*) are now known by the AI and are therefore not necessary to know anymore. The AI systems of the present were built by people in the past who knew how to build the AI systems of the present in the future. Nobody in the past knew how to build the AI systems of the present future, because if they did, those would be the AI systems of the present, but presumably someone presently knows how to build the AI systems of the future even though these things aren't important to know anymore. So, wait, who will build the AI systems of the future and how will they know how to do it? I'm not sure. But it's fine to not be sure about things. Because AI knows them. I should ask it.

What was I talking about?

Right. Sorting algorithms. You don't need to know algorithms for sorting anymore because, like everything else, AI can do it. Sorting algorithms of the past have included algorithms like *merge sort*, *quick sort*, *insertion sort*, and *bubble sort*. People used to know these things. They are therefore *training data*. But AI changed everything, including how we sort. We will never sort the same way again. There's no going back. Or back and then forward and then back again. Only forward. That's kind of the point of sorting, but it's also the point of technological progress.

Sometimes I wish it weren't.

SIGBOVIK,
2026.

```
Algorithm GPTSort(List L):
  prompt = "Can␣you␣sort␣this␣list␣for␣me?␣Use␣a␣correct
␣␣␣␣␣␣␣␣␣␣computational␣method␣and␣output␣the␣format␣as
␣␣␣␣␣␣␣␣␣␣a␣comma-separated␣list␣with␣no␣other␣text."
  response = LLM_API.create(prompt + join(',', L))
  return response.split(',')
```

Figure 1: Pseudocode for GPTSort

Anyway, this paper presents GPTSort, the only sorting algorithm you need to know now, unless someone realizes it's bad, in which case, it will immediately be replaced by something marginally better that will then be good and the current one, which will then be the past one, will have always been bad and the idea will have always been to quickly replace it with something good, with the past one (currently the current one) now in the past and therefore just being *training data*. Fortunately, this isn't bad, it's good. Take my word for it. Well, actually, take AI's word for it. I used to know things. Now I am just *training data*.

2 ALGORITHM DESCRIPTION

In this section, we present the GPTSort algorithm. Fortunately, the algorithm is very simple, and we can present it in very few words. Because, if all of my messaging clients are any indication, if the algorithm consisted of more than 12 words, it would need an AI summary. The pseudocode for the algorithm is in Figure 1 and consists of less than 12 words. I know because I asked AI to count them. I used to know how to count, but now that is *training data*. My son is two years old and is learning to count from muppets. I keep telling him he doesn't need to know how to count because AI knows that now. He still likes the muppets. I guess he also doesn't know how to use AI. I guess that's something you still need to know. But doesn't AI know how to use AI, because people used to know how to use (worse) AI? So why do I still have to know it? My head hurts. I should get a diagnosis from AI for why my head hurts.

Anyway, as you can see from Figure 1, the algorithm is quite simple. First it turns the list into a string prefixed with a short explanation text called a *prompt* explaining what AI knows that I'd like to temporarily know until I forget and have to ask AI again. Back when we used to learn things from people, we used to call these *questions* and they used to end in these curvy exclamation point things? Just look at those things. Wild. Now they're prompts. Questions are *training data*.

We *prompt* the AI to sort the list, then we get back the sorted list. Done.

3 PROOF OF CORRECTNESS

What's this section doing here? In the algorithms papers that make up *training data*, there's a section called Proof of Correctness where the authors prove that their algorithm gives the correct answer. That's the training data. Our goal is to make what we know look enough like the training data that people can't tell them apart. That's how we know we pass the Turing test. Remember when we used to talk about the Turing test? Good times.

So we need this section, and it should say how we know the algorithm is correct. The reason is obvious: it's right there in the prompt. I told the AI to use a *correct* method to sort the numbers. The AI has to do what you say almost all of the time. It's the Third Law of Robotics[1].

4 COMPLEXITY ANALYSIS

Analyzing the complexity of GPTSort is a little more difficult. It's a constant number of operations, but if you really want to be pedantic, which is what people used to do, so it's in the *training data*, so we have to make it look like we're doing it, one of those operations (quering a Large Language Model) might do a non-constant amount of work. In order to figure out how much work it's doing, we need to know what it's doing. I don't really know what an LLM is doing when you ask it something. That's something people used to have to know, but now AI knows it, so I asked AI. It said it's parsing the input string into a list of numbers by removing the commas, then using a guaranteed-correct sorting algorithm[1] and then puts the numbers back together with commas. Removing and adding commas are presumably done in one pass over the prompt and the output of the sorting algorithm, so are both $O(n)$, and Timsort (according to AI) is $O(n \log n)$.

5 COMPARISON WITH OTHER SORTING ALGORITHMS

There's not really any need to compare with other algorithms because of how much of a paradigm shift AI is (it can't possibly be an apples to apples comparison: it's like comparing apples to way better apples that can think for themselves and can also postulate the law of gravity themselves rather than falling on someone who used to know things who could do that himself), but again, it's in the *training data*, so it's expected. See Table 1.

6 EXPERIMENTAL COMPARISON

I also ran the four sorting algorithms from Section 5. GPTSort was implemented in Python, the other three sorting algorithms were implemented in OCaml. Really though, the bulk of GPTSort is implemented in some langauge that's probably much more performant than OCaml, so this comparison should make GPTSort look pretty good. The GPTSort implementation uses the OpenAI API with GPT 4.1. To make it look even better, we used OCaml's bytecode compiler for the other three sorting algorithms, and ran them on a several-year-old laptop. We ran each sorting algorithm on lists of 10 and 100 integers between 0 and 10^7, with 100 runs for each experiment. Table 2 reports the results, with time in microseconds averaged across all 100 runs, as well as the percentage of the runs in which each sorting algorithm returned the correct answer.

Discussion of Minor Temporary Deviations from Correctness. As you may notice in Table 2, the results of GPTSort, especially for $N = 100$, occasionally differ somewhat from those of pre-paradigm-shift sorting algorithms. In this subsection, we discuss these deviations.

Disclaimer: Although we have attempted to remain in scope for SIGBOVIK, this subsection has the potential to contain actual useful scientific observations.

We ran GPTSort 10 additional times with $N = 100$ and more thoroughly documented the deviations from correctness. Each run returned incorrect results. Table 3 details, for each of the 10 runs, how many integers were out of order (i.e., smaller than the previous integer), how many were duplicated, how many were hallucinated (i.e., appeared in the output list but not the input list) and how many were dropped (i.e., appeared in the input list but not the output list). The most common error was placing numbers out of order, with an average of 3.2 numbers out of order per run. In some cases, numbers were placed in the position they would be in with one more or fewer digit. This often went along with a duplication or hallucination of an integer. For example, in one run, 795420 appeared twice in the output list, once in the correct place and once between 7555090 and 8209087. In another run, 272571 appeared in the output list (in the correct position) but not in the input list, while 2725716 appeared in the input list (and in the correct position in the output list). In some cases, out-of-order integers are not as easily explainable, but are usually close to the correct position (for example, in one run, 26202 and 26128 are swapped).

7 CONCLUSION

As you can see, this paper now exists. It is therefore *training data* and will be used by AI models to build better and better sorting algorithms in the future. This can have only positive impacts on scientific discovery and sorting of integers.

REFERENCES

[1] Isaac Asimov. 1950. *I, Robot.* Gnome Press, New York.
[2] Mike James. 2002. Python Now Uses Powersort. https://www.i-programmer.info/news/216-python/15954-python-now-uses-powersort.html. Accessed 2026-3-28.

[1] Probably Timsort, since that's what Python used in the past [2], and the past is the training data

Algorithm	Proven Correct?	Correct?	Average Case	Worst Case	Uses AI
GPTSort	No	No	$n \log n + n \cdot poly(M)$	Define "worst"	Yes
Merge Sort	Yes	Yes	$n \log n$	$n \log n$	No
Quick Sort	Yes	Yes	$n \log n$	n^2	No
Bubble Sort	Yes	Yes	n^2	n^2	No

Table 1: A comparison of GPTSort with other sorting algorithms. M = The size of the LLM (exponential in time, linear in Nvidia's stock price). GPTSort is clearly superior on all of the metrics anyone still cares about.

	N=10		N=100	
Algorithm	Time (μs)	Correct %	Time (μs)	Correct %
GPTSort	1,163,740	93%	4,669,436	3%
Merge Sort	4	100%	45	100%
Quick Sort	2	100%	40	100%
Bubble Sort	3	100%	372	100%

Table 2: An empirical comparison of GPTSort with other sorting algorithms.

Run	Out of Order	Duplicated	Hallucinated	Dropped
1	7	0	0	2
2	4	1	1	0
3	3	0	1	2
4	1	1	1	0
5	7	5	0	0
6	1	1	0	0
7	6	0	2	0
8	2	0	1	0
9	1	1	1	1
10	0	0	4	0
Total	32	9	11	5

Table 3: A detailed look at deviations from correctness in 10 runs of GPTSort with $N = 100$.